

DD2356 VT26 Project Report

Pengyu Wang's Part: MPI, Hybrid MPI+OpenMP, and GPU Offloading

Group 26

1 Design and Implementation

1.1 MPI Decomposition and Communication

The MPI implementation uses a destination-node block decomposition. For a graph with N vertices and P MPI ranks, rank r owns a contiguous vertex interval $[start_r, end_r)$ and computes only the corresponding entries of the next PageRank vector. The graph is stored in the same incoming-edge CSR layout as the serial baseline: `row_ptr[v]` to `row_ptr[v+1]-1` enumerates all source vertices contributing to destination vertex v . To keep the first distributed-memory version simple and easy to verify, each rank stores the full CSR graph and the full current PageRank vector, while only the newly computed block `pr_new[start:end]` is rank-local.

The graph is loaded on rank 0 and distributed to the other ranks with `MPI_Bcast`. Each PageRank iteration then performs the following communication:

1. One scalar `MPI_Allreduce` for the global dangling-node mass.
2. One scalar `MPI_Allreduce` for the global L_1 convergence difference.
3. One `MPI_Allgatherv` to assemble the full next PageRank vector on every rank.

The per-iteration cost can therefore be modeled as

$$T_{\text{iter}}(P) \approx T_{\text{comp}} \left(\frac{M}{P} \right) + 2T_{\text{allreduce}}(P) + T_{\text{allgatherv}}(N, P),$$

where M is the number of edges. This model predicts that the implementation will be communication-bound for small graphs or when the full-vector `Allgatherv` dominates the reduced local sparse update.

1.2 Hybrid MPI+OpenMP

The hybrid version keeps the same MPI node-block decomposition and adds OpenMP parallelism inside each MPI rank. The dangling-node mass and convergence checks use OpenMP reductions, while the sparse incoming-edge PageRank update uses `#pragma omp parallel for schedule(dynamic, 256)`. The dynamic schedule was chosen because real graphs such as `polblogs.csv` have highly irregular in-degree distributions; assigning a fixed number of destination vertices to each thread can leave some threads with many more incoming edges than others.

The profiling script supports fixed-core experiments with configurations `1x16`, `2x8`, `4x4`, `8x2`, and `16x1`, where the first number is the MPI rank count P and the second number is the OpenMP thread count N per rank. These runs are intended to identify whether the best performance comes from fewer MPI ranks with more threads, or more MPI ranks with less intra-rank threading.

1.3 OpenMP GPU Offloading

The GPU comparison path is implemented with OpenMP target offloading. The dangling-node reduction and sparse PageRank update are expressed with `target teams distribute parallel for`; the convergence norm remains on the CPU in the current prototype. The implementation maps the PageRank vector, CSR arrays, and precomputed inverse out-degree array to the device for each offloaded kernel. This is intentionally a correctness-first prototype, useful for comparing the cost of offloading against the best CPU paths before adding more aggressive persistent device-data regions.

2 Correctness Evaluation

All parallel outputs were compared against the serial PageRank reference using the repository verifier and a tolerance of 10^{-6} . The MPI implementation passed the full nine-dataset correctness suite on Dardel for $P = 1, 2, 4$ ranks, and the Dardel scaling experiments passed for all measured $P = 1, 2, 4, 8, 16$ cases. On the school cluster, the MPI strong-scaling runs for both `polblogs.csv` and `synthetic_100k_1m.csv`, as well as the weak-scaling sequence up to the 200k-node/2M-edge graph, passed for all measured rank counts. The hybrid fixed-core runs passed for every tested (P, N) combination on both datasets. The maximum errors are at machine precision scale, e.g. at most 1.04×10^{-17} in the Dardel `polblogs.csv` MPI run.

Table 1: Correctness summary for Wang-owned implementations.

Implementation	Dataset scope	Configuration	Result
MPI	9 course datasets	1, 2, 4 ranks on Dardel	PASS, tol. 10^{-6}
MPI	Dardel scaling datasets	1, 2, 4, 8, 16 ranks	PASS, tol. 10^{-6}
MPI	2 cluster datasets	1, 2, 4, 8, 16 ranks on school cluster	PASS, tol. 10^{-6}
MPI	cluster weak-scaling sequence	1 to 16 ranks; up to 200k nodes / 2M edges	PASS, tol. 10^{-6}
Hybrid MPI+OpenMP	2 cluster datasets	fixed 16 cores: 1x16 to 16x1	PASS, tol. 10^{-6}
OpenMP target GPU	9 course datasets	local target-offload prototype	PASS vs. serial

3 Scaling Evaluation

3.1 Dardel MPI Strong Scaling

Table 2 reports the Dardel strong-scaling result for the course `polblogs.csv` graph. The graph is small (1224 vertices and 19090 edges), so the communication overhead is much larger than the useful local sparse update once more than one rank is used.

Table 2: MPI strong scaling on Dardel for `polblogs.csv` (directed).

Ranks	Runs	PR time (s)	Comm frac.	Speedup	Efficiency	Imbalance
1	5	0.002560	0.0430	1.000000	1.000000	1.000
2	5	7.836485	0.9997	0.000327	0.000163	1.887
4	5	8.291886	0.9998	0.000309	0.000077	3.129
8	5	11.579997	0.9999	0.000221	0.000028	4.412
16	5	14.117646	1.0000	0.000181	0.000011	4.536

To separate graph-size effects from implementation effects, the same experiment was repeated on a larger synthetic graph with 100000 vertices and 1000000 edges. The larger graph has much better block balance (max/average in-edge imbalance around 1.05), but Table 3 still shows negative scaling after one rank. This confirms that the bottleneck is dominated by collective communication rather than by local load imbalance.

Table 3: MPI strong scaling on Dardel for `synthetic_100k_1m.csv` (directed).

Ranks	Runs	PR time (s)	Comm frac.	Speedup	Efficiency	Imbalance
1	5	0.036353	0.0127	1.000000	1.000000	1.000
2	5	8.682688	0.9961	0.004187	0.002093	1.035
4	5	7.217915	0.9976	0.005037	0.001259	1.044
8	5	10.115917	0.9989	0.003594	0.000449	1.047
16	5	16.303321	0.9996	0.002230	0.000139	1.051

The communication fraction rises to approximately 0.996–0.999 for $P > 1$ on the synthetic graph. In the timing breakdown, most of this time is spent in the full-vector `MPI_Allgatherv` and the synchronization waits inside the scalar reductions. Therefore, this implementation is correct and useful as a distributed-memory baseline, but it does not provide scalable performance without reducing the amount of vector communication per iteration.

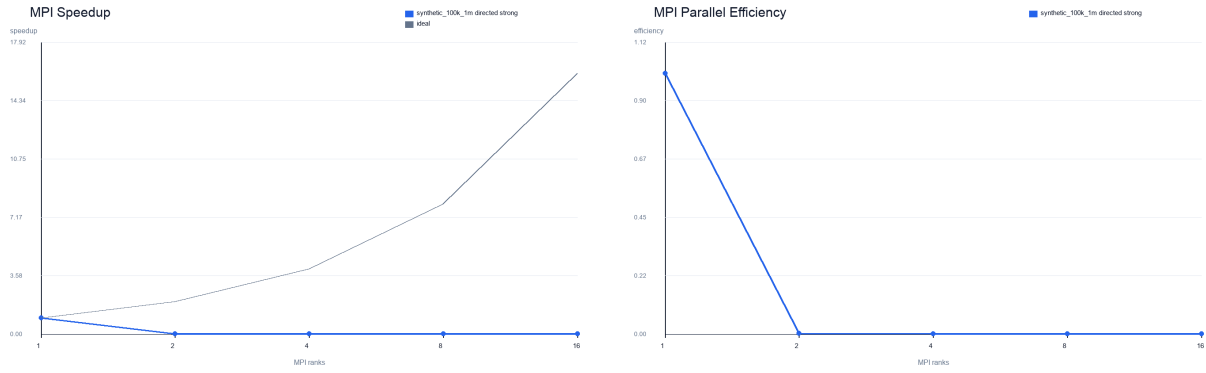


Figure 1: MPI speedup and parallel efficiency from the generated result figures.

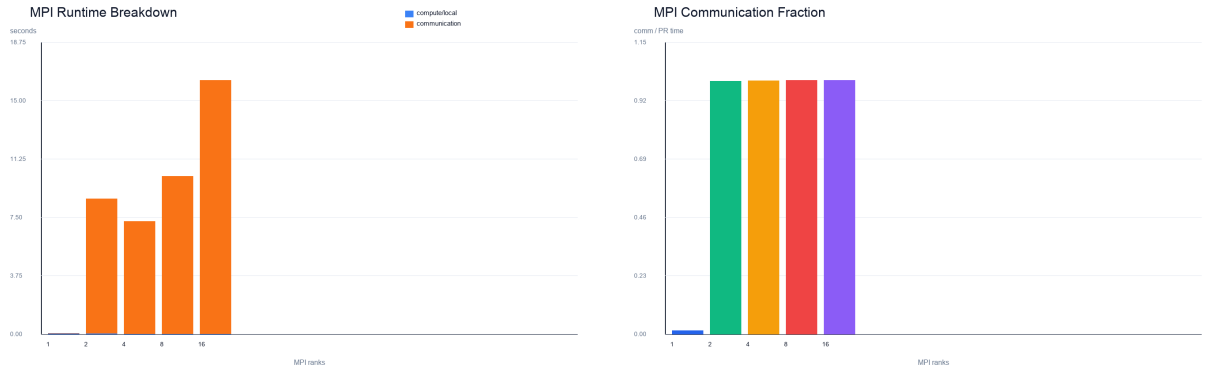


Figure 2: MPI runtime breakdown and communication fraction.

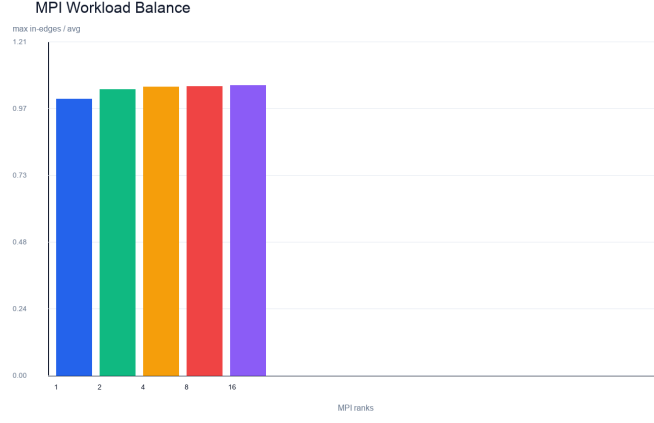


Figure 3: MPI workload imbalance measured as max in-edges divided by average in-edges.

3.2 Dardel MPI Weak Scaling

Weak scaling keeps the per-rank graph size approximately fixed while increasing the number of ranks. For this project I used 12500 vertices and 125000 edges per rank, so the required 16-rank experiment uses a 200000-vertex, 2000000-edge synthetic graph. This is a different experiment from strong scaling: strong scaling fixes the graph size, while weak scaling grows the global graph so each rank receives similar local work.

Table 4: MPI weak scaling on Dardel with 125k edges per rank.

Dataset	Ranks	Nodes	Edges	PR time (s)	Comm frac.	Weak eff.	Status
weak_1rank_12500_125000	1	12500	125000	0.003857	0.0338	1.000000000	PASS
weak_2rank_25000_250000	2	25000	250000	2.004018	0.9978	0.001924534	PASS
weak_4rank_50000_500000	4	50000	500000	2.101621	0.9975	0.001835155	PASS
weak_8rank_100000_1000000	8	100000	1000000	9.919046	0.9987	0.000388828	PASS
weak_16rank_200000_2000000	16	200000	2000000	23.825432	0.9992	0.000161877	PASS

The 16-rank row satisfies the required weak-scaling case and confirms that the dynamic graph reader can process the larger 200k-node input. However, weak efficiency drops sharply because the replicated-vector algorithm still communicates the full global PageRank vector every iteration. In the 16-rank weak-scaling run, MPI_Allgather alone averages 21.812665 s out of 23.825432 s PageRank time, so increasing local work per rank is not enough to hide the global communication cost.

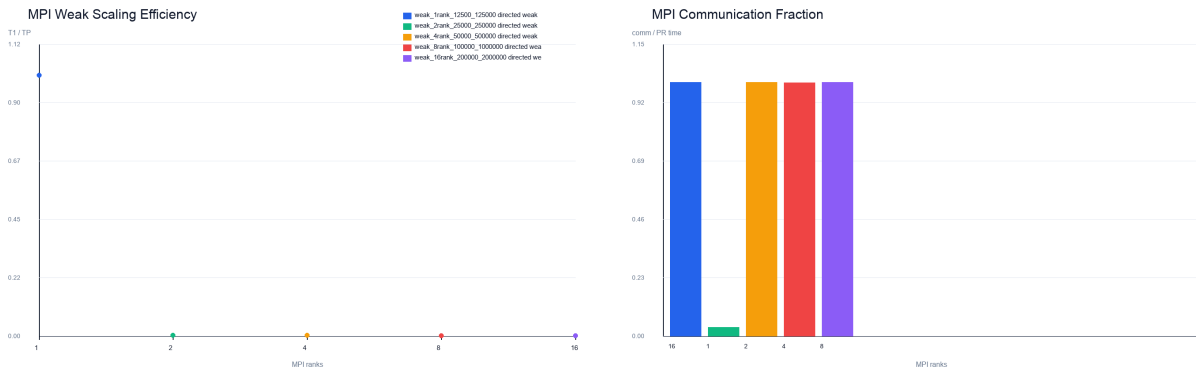


Figure 4: Dardel weak-scaling efficiency and communication fraction.

3.3 School-Cluster MPI Strong Scaling

The same MPI profiler was run on the school cluster. Table 5 shows that the small `polblogs.csv` graph remains communication-bound: the one-rank average PR time is only 0.001484 s, and adding ranks increases the communication fraction to 0.84–0.99. This is the expected regime where collective synchronization dominates useful work.

Table 5: MPI strong scaling on the school cluster for `polblogs.csv` (directed).

Ranks	Runs	PR time (s)	Comm frac.	Speedup	Efficiency	Imbalance
1	5	0.001484	0.0142	1.000000	1.000000	1.000
2	5	0.001889	0.8397	0.785472	0.392736	1.887
4	5	0.002927	0.8878	0.506936	0.126734	3.129
8	5	0.002461	0.9801	0.602844	0.075356	4.412
16	5	0.002844	0.9898	0.521733	0.032608	4.536

For the larger synthetic graph, the school-cluster MPI run scales up to four ranks before communication dominates. Table 6 shows the best average speedup at $P = 4$ with $2.14\times$ speedup and 0.534 parallel efficiency. From $P = 8$ onward, the communication fraction rises from 0.44 to 0.75 and then 0.85, so the extra ranks no longer compensate for the collective communication cost. The raw rank-16 measurements include one large outlier (PR time = 0.070486 s); therefore the table keeps the summary average as the main metric while the median rank-16 time (0.012609 s) is useful as a robustness check.

Table 6: MPI strong scaling on the school cluster for `synthetic_100k_1m.csv` (directed).

Ranks	Runs	PR time (s)	Comm frac.	Speedup	Efficiency	Imbalance
1	5	0.027228	0.0137	1.000000	1.000000	1.000
2	5	0.017646	0.1766	1.543006	0.771503	1.035
4	5	0.012738	0.4356	2.137590	0.534398	1.044
8	5	0.015967	0.7505	1.705301	0.213163	1.047
16	5	0.023347	0.8478	1.166240	0.072890	1.051

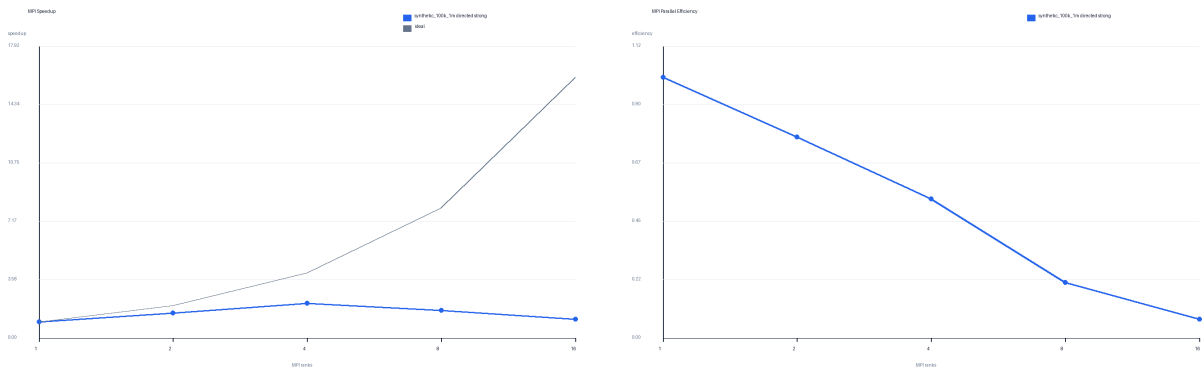


Figure 5: School-cluster MPI speedup and parallel efficiency for `synthetic_100k_1m.csv`.

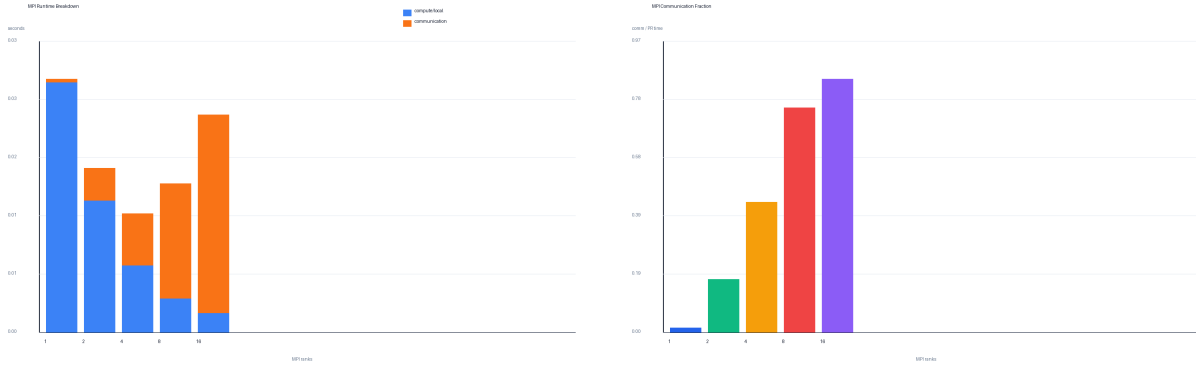


Figure 6: School-cluster MPI runtime breakdown and communication fraction.

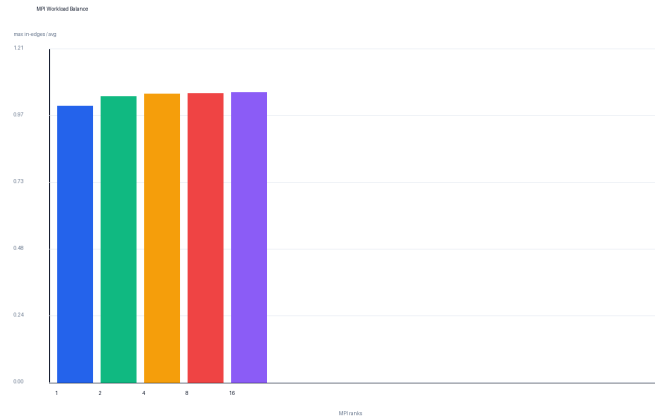


Figure 7: School-cluster MPI workload imbalance for `synthetic_100k_1m.csv`.

3.4 School-Cluster MPI Weak Scaling

The school-cluster weak-scaling experiment used the same generated graph instances as the Dardel experiment, with 125000 edges assigned per rank and a 16-rank maximum input of 200000 vertices and 2000000 edges. Equality of the per-rank in-edge distributions in the two result CSV files confirms that the comparisons are made on identical graph instances.

Table 7: MPI weak scaling on the school cluster with 125k edges per rank.

Dataset	Ranks	Nodes	Edges	PR time (s)	Comm frac.	Weak eff.	Status
<code>weak_1rank_12500_125000</code>	1	12500	125000	0.003310	0.0123	1.00000000	PASS
<code>weak_2rank_25000_250000</code>	2	25000	250000	0.005345	0.2442	0.619298836	PASS
<code>weak_4rank_50000_500000</code>	4	50000	500000	0.007542	0.4821	0.438951947	PASS
<code>weak_8rank_100000_1000000</code>	8	100000	1000000	0.011587	0.5887	0.285694560	PASS
<code>weak_16rank_200000_2000000</code>	16	200000	2000000	0.024371	0.6814	0.135833573	PASS

Although the school cluster performs substantially better than the measured Dardel job, its weak efficiency still falls from 1.0 to 0.1358 by 16 ranks. The per-rank incoming-edge imbalance remains small (at most 1.051), while communication consumes 68.14% of PageRank time at 16 ranks. Therefore, the weak-scaling deterioration is attributable primarily to the replicated-vector collectives, rather than to uneven sparse work assignment.

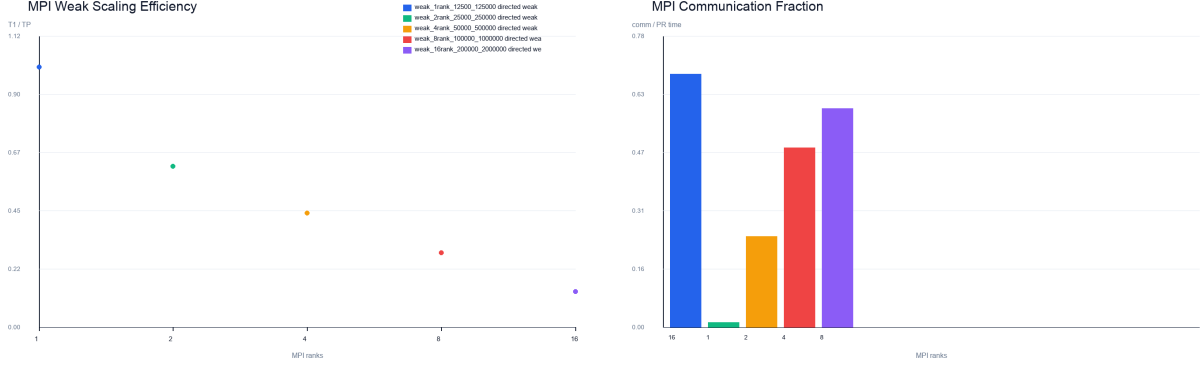


Figure 8: School-cluster weak-scaling efficiency and communication fraction.

4 Optimization Effectiveness

The first optimization is to precompute `inv_out_degree[u]` during graph construction. The PageRank inner loop then uses multiplication instead of performing a floating-point division for every incoming edge contribution:

$$s_v = \sum_{u \in In(v)} pr_u \cdot \text{inv_out_degree}[u].$$

This directly targets the sparse-update kernel, which is the dominant computation in the serial and single-rank paths.

The second optimization is the hybrid sparse-loop schedule. Because destination vertices can have very different numbers of incoming edges, `schedule(dynamic, 256)` reduces thread-level imbalance compared with a pure static assignment. This does not remove MPI collective overhead, but it improves the local work distribution within each rank for irregular graphs.

To verify these two choices directly, I added an ablation profiler that builds four hybrid variants: `no_inv_static`, `inv_static`, `no_inv_dynamic`, and `inv_dynamic`. The intended comparison is run at fixed total workers with 1x16 and 4x4. The effectiveness metrics are computed as

$$S_{\text{inv}} = T_{\text{no_inv_static}}/T_{\text{inv_static}},$$

$$S_{\text{dyn}} = T_{\text{inv_static}}/T_{\text{inv_dynamic}}.$$

and the full optimization speedup is $T_{\text{no_inv_static}}/T_{\text{inv_dynamic}}$. Table 8 reports the measured school-cluster averages from the two `results/optimization_ablation_cluster_*.csv` summary files; all variants passed verification against the serial reference.

Table 8: School-cluster optimization ablation, PR time in seconds.

Dataset	Config	No-inv static	Inv static	No-inv dyn.	Inv dyn.	Full speedup
synthetic_100k_1m	1x16	0.035235	0.033692	0.038780	0.030837	1.1426
synthetic_100k_1m	4x4	0.012647	0.014116	0.012424	0.010092	1.2532
polblogs	1x16	0.028250	0.023086	0.028299	0.026190	1.0787
polblogs	4x4	0.005697	0.008286	0.006149	0.005961	0.9557

The ablation results are mixed but useful. On the larger synthetic graph, the fully optimized variant is best for both tested configurations: it improves 1x16 by $1.1426\times$ and 4x4 by $1.2532\times$ over the unoptimized static/division version. In the 4x4 case, dynamic scheduling is the dominant local optimization, improving `inv_static` to `inv_dynamic` by $1.3987\times$. On `polblogs.csv`, the graph is small enough that timing noise and MPI collective overhead hide some local kernel gains: precomputing `inv_out_degree` helps the single-rank 1x16 case ($1.2237\times$), while the full 4x4 optimization is slightly slower than the unoptimized baseline ($0.9557\times$).

The GPU offloading prototype was correct on all course datasets, but it was not faster on the measured local `polblogs.csv` case. The local serial PR time was 0.001943 s, while the OpenMP target version took 0.053566 s. The best school-cluster hybrid result for `polblogs.csv` was also much lower at 0.0029068 s. This is not an apples-to-apples hardware comparison, but it shows that the current graph is too small for GPU offloading to amortize target launch and data mapping overheads, especially because the prototype maps arrays for each target region instead of keeping device data resident across the full iteration loop.

Table 9: Local GPU comparison on `polblogs.csv` (directed).

Variant	Configuration	PR time (s)	Correctness
Serial	1 thread	0.001943	reference
OpenMP target GPU	target offload prototype	0.053566	PASS

For hybrid MPI+OpenMP, the fixed-core experiments keep the total number of CPU workers at 16 and vary the split between MPI ranks and OpenMP threads. The reported speedup is relative to the first fixed-core configuration 1x16; it is not a speedup over the serial baseline.

Table 10: Hybrid fixed-core sweep on the school cluster for `polblogs.csv` (directed).

Ranks	Threads/rank	PR time (s)	Comm frac.	Speedup vs. 1x16	Status
1	16	0.024310	0.0018	1.0000	PASS
2	8	0.010850	0.1643	2.2406	PASS
4	4	0.005325	0.7428	4.5652	PASS
8	2	0.004281	0.8655	5.6783	PASS
16	1	0.002907	0.9533	8.3631	PASS

Table 11: Hybrid fixed-core sweep on the school cluster for `synthetic_100k_1m.csv` (directed).

Ranks	Threads/rank	PR time (s)	Comm frac.	Speedup vs. 1x16	Status
1	16	0.036692	0.0137	1.0000	PASS
2	8	0.018884	0.1758	1.9431	PASS
4	4	0.011843	0.6741	3.0981	PASS
8	2	0.019373	0.7682	1.8940	PASS
16	1	0.012234	0.8220	2.9992	PASS

The best hybrid split depends on graph size. For `polblogs.csv`, the best fixed-core result is 16x1, which avoids OpenMP overhead and benefits from the very small local work size despite high communication fraction. For `synthetic_100k_1m.csv`, the best average result is 4x4: it exposes enough MPI parallelism while keeping collective communication lower than the 8x2 and 16x1 configurations.

5 A-Level Coverage and Remaining Measurements

The new weak-scaling measurements close the previously missing 16-rank cases on both Dardel and the school cluster. However, the current evidence must not be interpreted as satisfying every A-level measurement requirement. In particular, the Dardel MPI measurements in this section were collected in a single-node allocation, whereas the requested criterion specifies multi-node Dardel strong and weak scaling.

Table 12: A-level evidence status after the current result consolidation.

Requirement	Evidence available	Still required
School-cluster MPI strong/weak scaling	Strong and weak CSVs; weak includes verified 16-rank 200k/2M case	Complete for measured rank sequence
Dardel MPI strong/weak scaling on multiple nodes	Verified strong and weak CSVs for 1–16 ranks on one shared node	Repeat scaling with ranks placed across at least two Dardel nodes
Hybrid fixed-core search on school cluster	Verified 1x16 through 16x1 sweeps	Repeat for additional total core budgets, not only $P \times N = 16$
OpenMP speedup on three systems	Correctness and local timing evidence	Provide scaling measurements on Colab, KTH, and Dardel
GPU offload comparison with Hybrid	Correct local offload prototype and separate school-cluster Hybrid timing	Obtain a same-platform GPU-vs-Hybrid timing comparison with confirmed device execution

Contribution

Pengyu Wang implemented the MPI decomposition and communication timing, ran and analyzed the Dardel and school-cluster MPI strong-scaling experiments, added the 16-rank weak-scaling workflow with dynamically sized graph readers, built and evaluated the hybrid MPI+OpenMP fixed-core profiling workflow, integrated the cluster result pipeline, and verified/compared the OpenMP GPU offloading prototype against the serial baseline.